Authentication vs Authorization

who are you? vs what may you do?

Watch the 4-min video

> **In one line:** authentication proves *who* the caller is, once, at the edge; authorization decides *what* that identity may do to *this specific resource*, on every request, in the service that owns the data.

1 Login worked. The bug is elsewhere

A user logs in, then calls `GET /orders/1042`. Order 1042 belongs to someone else, and the API returns it anyway. The password check was perfect; nobody checked **permission**. This is **Broken Object Level Authorization (BOLA)**, **API1** in the OWASP API Security Top 10 (2019 and 2023 editions), and the most common way real APIs leak data.

2 Two different questions

	Authentication (authN)	Authorization (authZ)
Question	Who are you?	May you do *this action* on *this resource*?
Input	password, passkey, one-time code, SSO assertion	verified identity + claims, the resource, the action
Output	an identity: session or token for user 42	allow or deny
How often	once per login; token verified per request	every request, every resource
Where	identity provider, gateway	the service that owns the data
HTTP on failure	401 Unauthorized (really “unauthenticated”)	403 Forbidden, or 404 to hide existence

Authentication produces an identity. Authorization *uses* that identity to make a decision, and it must make it again for every request, because each request touches a different resource.

3 Where each one lives

Authentication at the edge. A login service or identity provider (Entra ID, Okta, Auth0, Keycloak) checks the credentials and issues a session (lesson 40) or a signed token (lesson 41). The API gateway verifies that token on every request and rejects anonymous traffic early. The identity then travels inward as a user ID and claims; internal services never see passwords. The gateway must **strip** any identity headers that arrive from outside, so a client cannot simply send `X-User-Id: 1`.

Authorization close to the data. Only the order service knows who owns order 1042, so only it can decide. The gateway can do coarse checks that need no data; the fine-grained checks belong in the service.

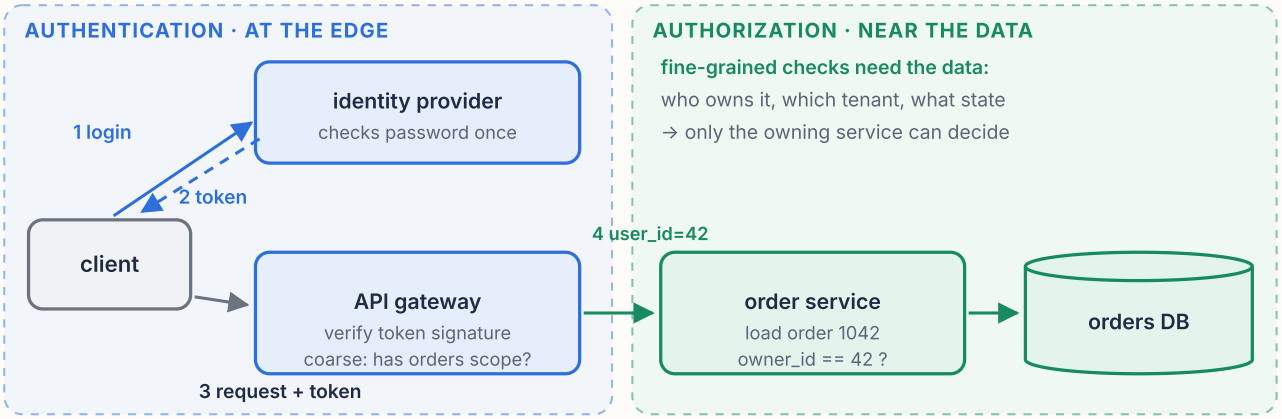


Figure 1. The identity provider checks credentials once; the gateway verifies the token on every call and forwards a trusted user ID; the order service, which can see `owner_id`, makes the final decision.

Gateway can check (coarse)

token valid: signature, issuer, expiry
scope present, e.g. orders:read
role claim, e.g. admin-only routes
rate limits per user or client

Service must check (fine-grained)

does user 42 own order 1042?
same tenant as the record?
is the record in an editable state?
field-level rules (hide cost price)

Figure 2. Split authorization by what each layer can see. Coarse checks at the gateway are an extra filter, never a substitute.

WORKED EXAMPLE: THE OWNERSHIP CHECK

User 42 signs in; the login service verifies the password and issues a token with `sub=42`. The user requests `GET /orders/1042`. The gateway validates the token's signature and expiry and forwards `user_id=42`. The order service runs `SELECT … WHERE id = 1042`, finds `owner_id = 77`, and returns **404 Not Found**. Authentication passed; authorization blocked the request. Better still, scope the query itself: `WHERE id = 1042 AND owner_id = 42`, so a missing check cannot leak a row.

4 Keep the jobs separate

GOOD PRACTICE

- Authenticate once, in one place, with a well-tested identity provider; never build your own password checks in every service.
- Pass a **verified** identity inward (signed token or gateway-set header on a trusted network), never raw passwords.
- **Deny by default**; authorize every request on the server against the specific resource, not just the endpoint.
- Centralise the rules in a policy function or engine (lesson 44) so every endpoint calls the same check.

COMMON MISTAKES

- Treating “has a valid token” as “may access every record”.
- Reading the role or user ID from the request body instead of from the verified token.
- Hiding a button in the UI and leaving the API endpoint open: anyone can call the API directly.
- Relying on unguessable IDs (UUIDs) instead of an ownership check; IDs leak through URLs, logs and shared links.

5 Key terms

Authentication: proving identity: who the caller is

Identity provider (IdP): service that authenticates users and issues tokens

BOLA / IDOR: accessing another user's object by changing its ID

Authorization: deciding what that identity may do

Claims: facts about the user in a token: sub, roles, scopes

Deny by default: anything not explicitly allowed is refused

6 Interview questions

Q1. Explain the difference between authentication and authorization, and where each is enforced. BASIC

Authentication proves who you are, usually once at the edge through an identity provider, with the gateway verifying the token per request. Authorization decides what that identity may do, checked on every request on the server, close to the data.

Q2. What is the difference between HTTP 401 and 403? BASIC

401 means the caller is not authenticated (missing or invalid credentials). 403 means the caller is known but not allowed; some APIs return 404 instead so they don't reveal that the record exists.

Q3. A tester changes /orders/1042 to /orders/1043 and sees another customer's order. How do you fix it? SCENARIO

Add an object-level check in the order service: load the order and compare its owner (and tenant) with the authenticated user, or scope the query by owner. Then add tests that call every endpoint with another user's IDs.

Q4. How does an internal service know which user made the request? SCENARIO

The gateway verifies the token and forwards the user ID and claims, either by passing the token on (each service verifies it) or in a header that the gateway sets after stripping any client-supplied copy. Services must not accept identity from anything the client controls.

Q5. What goes wrong if you mix up authentication and authorization? TRICKY

Logged-in users read or change other people's data: broken object level authorization, API1 in the OWASP API Security Top 10.

Q6. Do random UUIDs for order IDs fix BOLA? TRICKY

No. They make guessing harder, but IDs leak through links, logs and other APIs. Only a server-side check of ownership on every request fixes it.

Next: SD-M-002 · Session-Based Authentication backendroadmap.com